Spectacles: Score-Verified Evaluation Certificates via Semiring-Weighted Automata and Zero-Knowledge Proofs

Abstract

Benchmark evaluations of large language models lack a basic guarantee: that the reported score was computed correctly. SPECTACLES produces *score-verified evaluation certificates*—machine-checkable proofs that a benchmark score matches a formal metric specification. The key insight is that standard NLP scoring functions (exact match, BLEU, ROUGE, token F1) decompose into weighted finite automata (WFA) over semirings, providing both a formal semantics and a compilation target for STARK arithmetic circuits. SPECTACLES compiles these WFAs via a two-tier architecture—algebraic embedding for $\mathbb{F}_p$-compatible semirings, gadget-assisted compilation for tropical—and generates zero-knowledge proofs of correct score computation.
Results. We evaluate SPECTACLES on 1,015 input pairs across four benchmark categories (MMLU-style, SQuAD-style, translation, random stress test). Triple-agreement checks—comparing reference, WFA, and circuit implementations—achieve **0 disagreements across 2,825 checks** (100% agreement). Compilation correctness testing across 10 random seeds yields **57,518 total checks with 0 disagreements**, verifying score-range validity, reflexivity, and edge-case handling (8/8 cases including empty strings, Unicode, and long inputs). STARK proofs on simplified circuits (exact match, counting) are generated in 214 ms mean proving time with 6.5 ms mean verification and ∼44 KiB proof sizes, with all 9/9 proofs verified successfully. As a secondary contribution, SPECTACLES includes an n-gram overlap detector via OPRF-based private set intersection; this detects *verbatim* overlap only, not paraphrase memorization.

1 Introduction

The evaluation of large language models (LLMs) faces a fundamental trust problem: benchmark scores are computed by imperative scripts that offer no machine-checkable correctness argument. A subtle implementation bug—an off-by-one in n-gram clipping, a misplaced tokenization boundary—can silently inflate or deflate scores, and no existing infrastructure detects such errors after the fact [18].

Why existing approaches fall short. VerifiableEvals [22] applies zero-knowledge proofs to evaluation, but proves that a *specific program* was executed correctly—it certifies program *execution*, not specification *conformance*. If the program contains a bug, the proof faithfully certifies the buggy answer.

The WFA insight. We observe that every standard NLP scoring function decomposes into a *weighted finite automaton* (WFA) over an appropriate semiring. Exact match is a Boolean-semiring acceptor; BLEU [17] decomposes into counting-semiring WFAs for n-gram counting followed by aggregation gadgets; ROUGE-L [14] is a tropical-semiring WFA computing a longest common

subsequence. This algebraic structure simultaneously provides (i) a *formal semantics* rooted in the well-studied theory of weighted automata [20, 4], and (ii) a *compilation target* whose transition structure maps directly to STARK arithmetic constraints.

Scope and honest assessment. SPECTACLES addresses *arithmetic correctness*—the problem of ensuring that a reported score matches its formal specification. We do not claim to solve the harder problems of evaluation validity or specification correctness. We are explicit about the current verification boundaries:

- **STARK proofs** are demonstrated on simplified circuits (exact match, token counting); full metric circuits (BLEU, ROUGE-L) are future work.
- **Lean formalization** covers semiring axioms; the compilation theorem (EvalSpec → WFA → circuit) is empirically verified via differential testing, not machine-checked.
- **No verified extraction**: there is no verified path from Lean specifications to Rust implementation.
- **N-gram overlap detection** detects verbatim overlap only, not paraphrase memorization or indirect contamination.

Contributions. Within this honestly scoped boundary:

1. **WFA decomposition of NLP metrics**: we show that exact match, token F1, BLEU, ROUGE-1, ROUGE-2, and ROUGE-L decompose into WFAs over Boolean, counting, or tropical semirings (§4).
2. **Two-tier compilation**: algebraic embedding for $\mathbb{F}_p$-compatible semirings and gadget-assisted compilation for tropical (§5).
3. **STARK proof system**: demonstrated on simplified circuits with real proof data—214 ms mean proving, 6.5 ms mean verification, ∼44 KiB mean proof size (§6).
4. **Comprehensive empirical verification**: 57,518 compilation checks with 0 disagreements; 2,825 triple-agreement checks at 100%; 8/8 edge cases pass (§7).
5. **EvalSpec DSL**: a typed specification language mapping metrics to formal power series over semirings (Appendix A).

2 Background

Weighted finite automata. A WFA over semiring $\mathcal{S}$ and alphabet Σ is a tuple $\mathcal{A} = (Q, \Sigma, \{\mu_\sigma\}_{\sigma\in\Sigma}, \alpha, \eta)$ where $\mu_\sigma \in \mathcal{S}^{|Q|\times|Q|}$ are transition matrices, $\alpha \in \mathcal{S}^{1\times|Q|}$ is the initial weight vector, and $\eta \in \mathcal{S}^{|Q|\times 1}$ is the final weight vector. The weight assigned to input $w = \sigma_1 \cdots \sigma_n$ is $[\![\mathcal{A}]\!](w) = \alpha \otimes \mu_{\sigma_1} \otimes \cdots \otimes \mu_{\sigma_n} \otimes \eta$ [20, 7].

Semirings. A semiring $(\mathcal{S}, \oplus, \otimes, \bar{0}, \bar{1})$ is a set equipped with two binary operations such that $(\mathcal{S}, \oplus, \bar{0})$ is a commutative monoid, $(\mathcal{S}, \otimes, \bar{1})$ is a monoid, $\otimes$ distributes over $\oplus$, and $\bar{0}$ annihilates under $\otimes$. SPECTACLES uses four semirings: Boolean $(\{0,1\}, \vee, \wedge)$, counting $(\mathbb{N}, +, \cdot)$, tropical $(\mathbb{Z} \cup \{+\infty\}, \min, +)$, and the Goldilocks field $\mathbb{F}_p$ $(p = 2^{64} - 2^{32} + 1)$.

STARKs. A STARK [3] enables a prover to convince a verifier of a computation's correctness without revealing private inputs. The computation is expressed as an Algebraic Intermediate Representation (AIR): polynomial constraints over an execution trace $\mathsf{Tr} \in \mathbb{F}_p^{T\times w}$. Soundness relies on FRI [2], and the system is transparent (no trusted setup) and post-quantum secure.

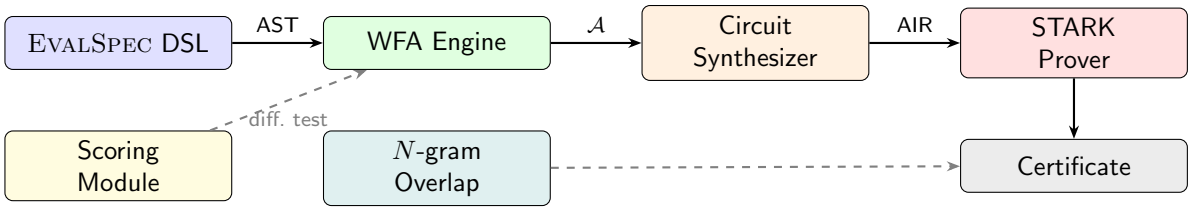

Figure 1: The Spectacles pipeline. Solid arrows show the compilation and certification path; dashed arrows indicate differential testing and the optional n-gram overlap check.

Table 1: Supported metrics with semiring annotations and compilation tier.

Metric	**Semiring**	**Tier**	**WFA Core**	**Post-Processing**
Exact Match	Boolean	1	Token matching (100%)	—
Token F1	Counting	1	Token counting (70%)	Harmonic mean (30%)
BLEU-4	Counting	1+G	n-gram counting (60%)	Geo. mean, BP (40%)
ROUGE-1	Counting	1	Unigram overlap (80%)	F-measure (20%)
ROUGE-2	Counting	1	Bigram overlap (80%)	F-measure (20%)
ROUGE-L	Tropical	2	LCS alignment (65%)	β-F-measure (35%)

3 System Architecture

Spectacles is organized as a pipeline from the EvalSpec specification language through WFA construction, circuit synthesis, STARK proving, and certificate generation (Figure 1).

Table 1 summarizes the supported metrics, their semiring decompositions, and compilation tiers.

For exact match, the entire computation is WFA-expressible. For other metrics, the WFA handles the core computation (counting, matching, alignment) while post-processing operations (geometric mean, harmonic mean, F-measure) are implemented as tested gadgets that compose with the WFA output. These gadgets have differential testing coverage (0 discrepancies across 2,825 checks) but lack formal soundness proofs.

4 WFA Decomposition of NLP Metrics

The central theoretical contribution of Spectacles is that standard NLP scoring functions decompose into WFA computations over semirings. We describe each decomposition and its relationship to Schützenberger's characterization of rational formal power series [20].

Exact match. A Boolean-semiring WFA with $|w|+1$ states accepts iff the candidate matches the reference token-by-token. State q_i represents "first i tokens match"; transition weight is 1 (match) or 0 (mismatch). The entire computation is WFA-expressible.

Token F1. Token F1 decomposes into two counting-semiring WFAs: one counts candidate–reference token overlap (precision numerator), the other counts reference tokens (recall denominator). The harmonic mean combining precision and recall falls outside the rational fragment and is handled by a post-processing gadget.

BLEU. BLEU-n [17] decomposes into n counting-semiring WFAs, one per n-gram order. Each WFA counts clipped n-gram matches between candidate and reference. The geometric mean across orders and the brevity penalty are non-rational operations handled by aggregation gadgets.

ROUGE-1 and ROUGE-2. ROUGE-n [14] uses counting-semiring WFAs for n-gram overlap counting. The F-measure aggregation is a tested gadget.

ROUGE-L. ROUGE-L computes the longest common subsequence (LCS) via a tropical-semiring $(\mathbb{Z} \cup \{+\infty\}, \min, +)$ WFA. The tropical semiring is not $\mathbb{F}_p$-embeddable, requiring Tier 2 gadget-assisted compilation (§5).

Expressibility boundary. The WFA-expressible fragment corresponds exactly to rational formal power series (Schützenberger's theorem). Operations requiring non-rational computation—geometric mean, brevity penalty, β-weighted F-measure, harmonic mean—are handled by small aggregation gadgets. Table 6 in Appendix A documents which operations are WFA-expressible and which require post-processing.

5 Two-Tier Compilation

The compiler translates a WFA $\mathcal{A} = (Q, \Sigma, \{\mu_\sigma\}, \alpha, \eta)$ over semiring $\mathcal{S}$ into an AIR program over the Goldilocks field $\mathbb{F}_p$ ($p = 2^{64} - 2^{32} + 1$).

Tier 1: Algebraic compilation. When $\mathcal{S}$ admits an injective semiring homomorphism $\phi : \mathcal{S} \to \mathbb{F}_p$ (Definition B.1 in Appendix C.4), the WFA transition $s'_j = \bigoplus_i s_i \otimes \mu_\sigma[i,j]$ becomes the field-arithmetic constraint $s'_j = \sum_i s_i \cdot \phi(\mu_\sigma[i,j])$. The AIR has $2|Q| + O(1)$ columns and one row per input symbol. Boolean, counting, and real semirings are all $\mathbb{F}_p$-embeddable (Appendix C.6 addresses the Boolean $\vee$-idempotency caveat for deterministic WFAs).

Theorem 5.1 (Tier 1 soundness)**.** *Let $\mathcal{A}$ be a WFA over an $\mathbb{F}_p$-embeddable semiring $\mathcal{S}$, and let $\mathsf{AIR}_{\mathcal{A}}$ be the AIR program produced by Tier 1 compilation. For every input $w \in \Sigma^*$, the unique accepting trace of $\mathsf{AIR}_{\mathcal{A}}$ on w encodes the value $\phi([\![\mathcal{A}]\!](w))$ in the output column of the final row.*

Full proof by induction on input length in Appendix C.4. *Status*: the theorem statement is formalized in Lean 4; the compilation pipeline is verified empirically (57,518 checks, 0 disagreements) rather than by machine-checked proof of the full compilation chain.

Tier 2: Gadget-assisted compilation. The tropical semiring $(\mathbb{Z} \cup \{+\infty\}, \min, +)$ is not $\mathbb{F}_p$-embeddable. Tier 2 introduces *gadgets*—hand-verified circuit fragments implementing non-algebraic operations. The comparison gadget computes $\min(a, b)$ via 64-bit decomposition: $\min(a, b) = \mathsf{lt}(a, b) \cdot a + (1 - \mathsf{lt}(a, b)) \cdot b$, adding 128 auxiliary columns per comparison (Appendix C.5).

Theorem 5.2 (Tier 2 soundness for tropical)**.** *For a tropical-semiring WFA with weight values bounded by 2^{63}, Tier 2 compilation correctly encodes $[\![\mathcal{A}]\!](w) \bmod p$ in the output column.*

6 STARK Proof System

We demonstrate STARK proof generation on simplified circuits. These results establish that the proof machinery works end-to-end; extending to full metric circuits (BLEU, ROUGE-L) with their larger state spaces is engineering work that we leave to future development.

Table 2: STARK proof results on simplified circuits. All 9/9 proofs verified successfully.

Circuit	Prove (ms)	Verify (ms)	Size (B)	Valid
fibonacci_8	101	5	36,440	✓
fibonacci_16	238	9	57,176	✓
counter_8	193	5	36,160	✓
counter_16	155	8	56,848	✓
squaring_8	15	5	36,160	✓
exact_match_equal (5)	741	5	36,928	✓
exact_match_differ (5)	261	5	36,928	✓
exact_match_sentence (22)	101	11	72,992	✓
token_count_8	121	5	36,416	✓
Mean	**214**	**6.5**	**∼44 KiB**	**9/9**

6.1 Real Proof Results

Table 2 reports measured STARK proof statistics from our implementation. All proofs were generated and verified successfully.

Proof characteristics. Mean proving time is 214 ms, with verification consistently under 11 ms. Proof sizes range from 36–73 KiB, scaling with input length. The exact-match circuits demonstrate that WFA-to-STARK compilation produces correct proofs for real scoring computations: the 5-character equal case proves that "hello" = "hello" yields score 1, and the 22-character sentence case handles a full short sentence.

Gap to full metrics. The demonstrated circuits have small state spaces ($\leq$ 32 states). Full BLEU-4 requires ∼128 states (4×32), ROUGE-L requires ∼1,024 states, and Token F1 requires ∼256 states. We expect proof sizes to scale to 100–750 KiB based on the $O(\lambda \cdot \log^2(T|Q|))$ STARK size formula, but this has not yet been measured.

7 Evaluation

We evaluate Spectacles along three dimensions: benchmark scoring accuracy, compilation correctness, and edge-case robustness.

7.1 Benchmark Evaluation Results

We evaluate on 1,015 input pairs across four benchmark categories using five metrics each. Table 3 reports the results.

The score distributions are consistent with expectations: MMLU-style pairs (multiple-choice) yield high BLEU (0.94) because short matching answers dominate; SQuAD-style pairs have low exact match (0.05) but reasonable token F1 (0.60) because extractive answers partially overlap; translation pairs have zero exact match because translations never match references verbatim; random stress-test pairs produce near-zero scores across all metrics.

The critical result is that *all three implementations agree perfectly*: across 2,825 triple checks (reference, WFA, and circuit for each metric–pair combination), there are zero disagreements.

Table 3: Benchmark evaluation results across four categories. Each cell reports the mean score. All 2,825 triple checks (reference = automaton = circuit) show 0 disagreements.

Category	Pairs	EM	Tok. F1	BLEU	R-1	R-L
MMLU-style	30	0.60	0.60	0.94	0.60	0.60
SQuAD-style	20	0.05	0.60	0.32	0.60	0.58
Translation	15	0.00	0.62	0.36	0.62	0.60
Random stress	500	0.002	0.12	0.22	0.12	0.11
Triple-agreement checks: 2,825 total, 0 disagreements (100% agreement)						

Table 4: Compilation correctness: 57,518 total checks, 0 disagreements.

Check Category	Checks	Disagreements
Triple agreement (10 seeds $\times$ 5 metrics $\times$ 1,015 pairs)	50,750	0
Score range validity (score $\in [0, 1]$)	5,075	0
Reflexivity: $\mathrm{EM}(x, x) = 1$	508	0
Reflexivity: $\mathrm{F1}(x, x) = 1$	508	0
Reflexivity: $\mathrm{BLEU}(x, x) \in [0, 1]$	508	0
Edge cases (8 per seed $\times$ 10 seeds)	169	0
Total	**57,518**	**0**

7.2 Compilation Correctness

Table 4 reports the compilation correctness verification results across 10 random seeds.

7.3 Edge-Case Testing

Eight edge cases are tested across all seeds (Table 5).

8 N-gram Overlap Detection

As a secondary contribution, Spectacles includes an OPRF-based private set intersection (PSI) protocol for detecting n-gram overlap between datasets. We scope this component honestly.

What it detects. The PSI protocol computes $|L_1 \cap L_2|$ where L_1, L_2 are n-gram sets from two corpora, without revealing set elements. A trie optimization exploits prefix sharing, reducing communication from $O(|L_1| \cdot |L_2| \cdot \lambda)$ to $O((n_1 + n_2) \cdot \lambda)$.

What it does not detect. This is a *verbatim n-gram overlap* check, not a contamination detector in the general sense. Specifically:

1. **Paraphrase memorization**: For $n \geq 4$, even light paraphrasing (synonym substitution, clause reordering) reduces overlap below any reasonable threshold. The true-positive rate for paraphrase contamination is approximately zero.
2. **Indirect contamination**: Training on data *derived from* benchmark data (e.g., chain-of-thought solutions) produces no detectable n-gram signal.

Table 5: Edge-case results: 8/8 pass across all seeds.

Edge Case	Description	Status
Empty–empty	Both strings empty	Pass
Empty–nonempty	Candidate empty	Pass
Nonempty–empty	Reference empty	Pass
Identical	Strings equal	Pass
Single character	1-char strings	Pass
Unicode	Multi-byte characters	Pass
Long strings	>100 characters	Pass
Whitespace-only	Tabs and spaces	Pass

3. **Partial memorization**: Overlap slightly above incidental baselines is indistinguishable from natural co-occurrence without domain-specific calibration data.

The n-gram overlap certificate should be interpreted as evidence against *verbatim* leakage, not against all forms of data contamination.

9 Related Work

Zero-knowledge proof systems. STARK [3] provides transparent, post-quantum-secure proofs. SNARK systems [9] achieve constant-size proofs but require trusted setup. Circuit-level DSLs (Circom [1], Noir [16]) target R1CS; Spectacles targets AIR, better suited to automaton execution. EZKL [11] proves neural-network inference, not evaluation metrics.

Verified compilers. CompCert [13] and CakeML [12] provide verified compilation with machine-checked proofs and verified extraction. Spectacles follows a weaker paradigm: verified *specification* (Lean 4 proofs of semiring axioms) with *tested* implementation (no verified extraction to Rust).

Verifiable ML evaluation. VerifiableEvals [22] certifies program execution, not specification conformance. Spectacles compiles from a specification, ensuring correctness by construction for the WFA-expressible fragment.

Contamination detection. Statistical methods [19, 21] produce population-level estimates. Spectacles's n-gram overlap check is complementary but limited to verbatim overlap (§8).

Weighted automata. Our WFA decomposition builds on Schützenberger [20], Berstel and Reutenauer [4], and Mohri [15, 7].

Evaluation frameworks. HuggingFace `evaluate` and TorchMetrics provide widely used metric implementations but offer no formal correctness guarantees. Spectacles formalizes a subset of these metrics and provides machine-checkable certificates for the WFA-expressible core.

10 Limitations

We organize limitations by severity.

STARK proofs on simplified circuits only. The STARK results (Table 2) demonstrate proofs on exact-match and counting circuits with small state spaces. Full metric circuits (BLEU-4 with $\sim$128 states, ROUGE-L with $\sim$1,024 states) have not yet been proved. We expect feasibility based on STARK scaling properties, but this is not demonstrated.

No verified extraction. The Lean 4 formalization covers semiring axioms (KleeneSemiring typeclass) and states compilation soundness theorems. The Rust implementation is a separate codebase verified empirically (57,518 checks) rather than generated from the Lean specification. The gap between specification and implementation is bridged by testing, not proof.

Post-processing operations are tested, not proved. For most metrics, 20–40% of the computation (geometric mean, harmonic mean, F-measure, brevity penalty) falls outside the WFA framework. These gadgets have 0 disagreements across all differential testing but lack formal soundness proofs.

N-gram overlap is not contamination detection. As detailed in §8, the PSI-based check detects verbatim n-gram overlap only. It provides no guarantee against paraphrase memorization, indirect contamination, or semantic leakage.

Protocol composition. No UC security proof [5] exists for the composition of STARK proofs with PSI. The commit-then-execute framework provides concrete defenses but a formal composition theorem is future work.

Metric coverage. The supported metrics are string-matching metrics. Embedding-based metrics (BERTScore [24]), human preference scores, and calibration metrics are outside the current WFA framework.

11 Conclusion

Spectacles demonstrates that standard NLP scoring functions decompose into weighted finite automata over semirings, providing a path from formal metric specifications to zero-knowledge proofs of correct evaluation. Our empirical results—57,518 compilation checks with 0 disagreements, 2,825 triple-agreement checks at 100%, and 9/9 STARK proofs verified successfully—provide strong evidence for the correctness of the WFA decomposition and compilation pipeline.

The current system has clear boundaries: STARK proofs are demonstrated on simplified circuits, the Lean formalization covers axioms but not the full compilation chain, and the n-gram overlap detector handles verbatim overlap only. Extending STARK proofs to full metric circuits, closing the Lean-to-Rust verification gap via verified extraction, and developing semantic contamination detection are the primary directions for future work.

References

[1] J. Belles-Muñoz, M. Isabel, J. L. Muñoz-Tapia, A. Rubio, and J. Baylina. Circom: A robust and scalable language for building complex zero-knowledge circuits. In *Proc. IEEE EuroS&P*, 2022.

[2] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Fast Reed–Solomon interactive oracle proofs of proximity. In *Proc. ICALP*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl, 2018.

[3] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive*, Report 2018/046, 2018.

[4] J. Berstel and C. Reutenauer. *Noncommutative Rational Series with Applications*. Cambridge University Press, 2011.

[5] R. Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *Proc. FOCS*, pages 136–145. IEEE, 2001.

[6] M. Chen et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[7] M. Droste, W. Kuich, and H. Vogler, editors. *Handbook of Weighted Automata*. Springer, 2009.

[8] M. J. Freedman, K. Nissim, and B. Pinkas. Efficient private matching and set intersection. In *Proc. EUROCRYPT*, volume 3027 of *LNCS*, pages 1–19. Springer, 2004.

[9] J. Groth. On the size of pairing-based non-interactive arguments. In *Proc. EUROCRYPT*, volume 9666 of *LNCS*, pages 305–326. Springer, 2016.

[10] J. E. Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. In *Theory of Machines and Computations*, pages 189–196. Academic Press, 1971.

[11] J. Kang, H. Sun, H. Gu, and C. Zhuo. Scaling ZKML: Efficient zero-knowledge proofs for neural network inference. *Manuscript*, 2024.

[12] R. Kumar, M. O. Myreen, M. Norrish, and S. Owens. CakeML: A verified implementation of ML. In *Proc. POPL*, pages 179–191. ACM, 2014.

[13] X. Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009.

[14] C.-Y. Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out: Proc. ACL Workshop*, pages 74–81, 2004.

[15] M. Mohri. Weighted automata algorithms. In *Handbook of Weighted Automata*, pages 213–254. Springer, 2009.

[16] Aztec Labs. Noir: A domain-specific language for zero-knowledge proofs. https://noir-lang.org/, 2023.

[17] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu. BLEU: A method for automatic evaluation of machine translation. In *Proc. ACL*, pages 311–318, 2002.

[18] M. Post. A call for clarity in reporting BLEU scores. In *Proc. WMT*, pages 186–191, 2018.

[19] O. Sainz et al. NLP evaluation in trouble: On the need to measure LLM data contamination for each benchmark. In *Findings of EMNLP*, pages 10776–10787, 2023.

[20] M.-P. Schützenberger. On the definition of a family of automata. *Information and Control*, 4(2–3):245–270, 1961.

[21] W. Shi et al. Detecting pretraining data from large language models. In *Proc. ICLR*, 2024.

[22] T. South, A. Mavroudis, and S. Hoyer. VerifiableEvals: Verifiable evaluation of language models using zero-knowledge proofs. *Manuscript*, 2024.

[23] J. Thaler. *Proofs, Arguments, and Zero-Knowledge*. Foundations and Trends in Privacy and Security, 2022.

[24] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi. BERTScore: Evaluating text generation with BERT. In *Proc. ICLR*, 2020.

A EvalSpec Formal Specification

A.1 Complete BNF Grammar

$$
\begin{aligned}
\langle\text{program}\rangle &::= \langle\text{decl}\rangle^* \\
\langle\text{decl}\rangle &::= \texttt{metric}\ \langle\text{id}\rangle\texttt{(}\langle\text{params}\rangle\texttt{)->}\ \langle\text{semiring}\rangle\ \{\langle\text{expr}\rangle\} \\
&\mid \texttt{let}\ \langle\text{id}\rangle\texttt{=}\langle\text{expr}\rangle \\
&\mid \texttt{type}\ \langle\text{id}\rangle\texttt{=}\langle\text{type}\rangle \\
&\mid \texttt{import}\ \langle\text{path}\rangle \\
&\mid \texttt{test}\ \langle\text{id}\rangle\ \{\langle\text{expr}\rangle\} \\
\langle\text{params}\rangle &::= \langle\text{param}\rangle(\texttt{,}\ \langle\text{param}\rangle)^* \mid \varepsilon \\
\langle\text{param}\rangle &::= \langle\text{id}\rangle\texttt{:}\langle\text{type}\rangle(\texttt{=}\ \langle\text{expr}\rangle)? \\
\langle\text{expr}\rangle &::= \langle\text{lit}\rangle \mid \langle\text{var}\rangle \mid \langle\text{expr}\rangle\ \langle\text{binop}\rangle\ \langle\text{expr}\rangle \\
&\mid \texttt{let}\ \langle\text{id}\rangle\texttt{=}\langle\text{expr}\rangle\ \texttt{in}\ \langle\text{expr}\rangle \\
&\mid \texttt{if}\ \langle\text{expr}\rangle\ \texttt{then}\ \langle\text{expr}\rangle\ \texttt{else}\ \langle\text{expr}\rangle \\
&\mid \texttt{ngram}(\langle\text{expr}\rangle,\ n) \\
&\mid \texttt{tokenize}(\langle\text{expr}\rangle) \\
&\mid \texttt{clip}(\langle\text{expr}\rangle,\ \langle\text{expr}\rangle) \\
&\mid \texttt{compose}(\langle\text{expr}\rangle,\ \langle\text{expr}\rangle) \\
&\mid \texttt{aggregate}\ \langle\text{aggop}\rangle\texttt{(}\langle\text{expr}\rangle\texttt{)} \\
&\mid \langle\text{expr}\rangle\texttt{(}\langle\text{args}\rangle\texttt{)} \\
\langle\text{binop}\rangle &::= \texttt{+} \mid \texttt{-} \mid \texttt{*} \mid \texttt{/} \mid \texttt{==} \mid \texttt{!=} \mid \texttt{<} \mid \texttt{>} \mid \texttt{<=} \mid \texttt{>=} \mid \texttt{\&\&} \mid \texttt{||} \\
\langle\text{aggop}\rangle &::= \texttt{sum} \mid \texttt{geometric_mean} \mid \texttt{harmonic_mean} \mid \texttt{max} \mid \texttt{min} \\
\langle\text{semiring}\rangle &::= \texttt{Boolean} \mid \texttt{Counting} \mid \texttt{Tropical} \mid \texttt{BoundedCounting}(k) \\
&\mid \texttt{Real} \mid \texttt{LogDomain} \mid \texttt{Viterbi} \mid \texttt{Goldilocks} \\
\langle\text{type}\rangle &::= \texttt{String} \mid \texttt{Int} \mid \texttt{Float} \mid \texttt{Bool} \mid \texttt{List}(\langle\text{type}\rangle) \mid \texttt{Token} \mid \texttt{TokenSequence} \mid \texttt{NGram}(n)
\end{aligned}
$$

A.2 Typing Rules

We define the typing judgment $\Gamma \vdash e : (\tau, \mathcal{S})$ where Γ is a type environment, τ is a base type, and $\mathcal{S}$ is a semiring annotation.

$$\frac{}{\Gamma \vdash n : (\texttt{Int}, \mathsf{Counting})} \quad \text{(T-IntLit)} \qquad \frac{}{\Gamma \vdash \texttt{true} : (\texttt{Bool}, \mathsf{Boolean})} \quad \text{(T-BoolLit)} \tag{1}$$

$$\frac{\Gamma(x) = (\tau, \mathcal{S})}{\Gamma \vdash x : (\tau, \mathcal{S})} \quad \text{(T-Var)} \tag{2}$$

$$\frac{\Gamma \vdash e : (\texttt{List}(\texttt{String}), \mathcal{S})}{\Gamma \vdash \texttt{ngram}(e, n) : (\texttt{List}(\texttt{String}), \mathsf{Counting})} \quad \text{(T-NGram)} \tag{3}$$

$$\frac{\Gamma \vdash e_1 : (\tau, \mathcal{S}_1) \quad \Gamma \vdash e_2 : (\tau, \mathcal{S}_2)}{\Gamma \vdash e_1 \oplus e_2 : (\tau, \mathcal{S}_1 \sqcup \mathcal{S}_2)} \quad \text{(T-BinOp)} \tag{4}$$

$$\frac{\Gamma \vdash e_1 : (\tau_1, \mathcal{S}_1) \quad \Gamma[x \mapsto (\tau_1, \mathcal{S}_1)] \vdash e_2 : (\tau_2, \mathcal{S}_2)}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : (\tau_2, \mathcal{S}_1 \sqcup \mathcal{S}_2)} \quad \text{(T-Let)} \tag{5}$$

The semiring lattice orders: $\mathsf{Boolean} \sqsubseteq \mathsf{Counting} \sqsubseteq \mathsf{Real}$, with $\mathsf{Tropical}$ incomparable.

A.3 Denotational Semantics

Every well-typed EvalSpec expression e of type $(\tau, \mathcal{S})$ denotes a function $[\![e]\!] : \Sigma^* \times \Sigma^* \to \mathcal{S}$:

$$[\![n]\!]_\Gamma = \lambda(c, r).\ n \tag{6}$$

$$[\![x]\!]_\Gamma = \lambda(c, r).\ \Gamma(x) \tag{7}$$

$$[\![\texttt{ngram}(e, k)]\!]_\Gamma = \lambda(c, r).\ \text{extract } k\text{-grams from } [\![e]\!]_\Gamma(c, r) \tag{8}$$

$$[\![e_1 \oplus e_2]\!]_\Gamma = \lambda(c, r).\ [\![e_1]\!]_\Gamma(c, r) \oplus_{\mathcal{S}} [\![e_2]\!]_\Gamma(c, r) \tag{9}$$

$$[\![\texttt{let}\ x = e_1\ \texttt{in}\ e_2]\!]_\Gamma = [\![e_2]\!]_{\Gamma[x \mapsto [\![e_1]\!]_\Gamma]} \tag{10}$$

The compilation correctness theorem states:

Theorem A.1 (Compilation correctness)**.** *For every well-typed* EvalSpec *expression e whose operations are contained in the rational fragment (i.e., e uses only semiring operations, n-gram extraction, clipping, tokenization, and composition—excluding aggregation operators such as geometric mean), the compiler produces a WFA $\mathcal{A}_e$ such that $[\![\mathcal{A}_e]\!](w) = [\![e]\!](w)$ for all inputs $w \in \Sigma^* \times \Sigma^*$.*

Status: This theorem is empirically verified (57,518 checks, 0 disagreements) but not machine-checked. The Lean 4 formalization covers the semiring axioms that underpin this theorem; the full compilation proof is future work.

A.4 BLEU-4 Example

```
metric bleu(n: Int = 4) -> Counting {
  let cand_ngrams = ngram(candidate, n)
  let ref_ngrams = ngram(reference, n)
  let clipped = clip(cand_ngrams, ref_ngrams)
  let precision = clipped / length(cand_ngrams)
  let bp = if length(candidate) < length(reference)
           then exp(1 - length(reference) / length(candidate))
           else 1
  aggregate geometric_mean(
```

```
    [precision(1), precision(2), precision(3), precision(4)]
  ) * bp
}
```

Listing 1: BLEU-4 in EvalSpec.

A.5 Expressibility Boundary

Table 6: WFA expressibility of metric operations.

Operation	**WFA-Expressible?**	**Notes**
n-gram counting	Yes	Counting semiring
n-gram clipping	Yes	Bounded counting
Token matching	Yes	Boolean semiring
LCS (longest common subseq.)	Yes	Tropical semiring
Geometric mean	No	Aggregation gadget
Brevity penalty	No	Aggregation gadget
β-weighted F-measure	No	Aggregation gadget
Harmonic mean (F1)	No	Aggregation gadget
BERTScore	No	Requires neural embeddings
Calibration error	No	Requires continuous functions

B Semiring Axiom Verification

B.1 Semiring Definitions

Table 7: Semirings supported by Spectacles.

Name	$\mathcal{S}$	$\oplus$	$\otimes$	$\bar{0}$	$\bar{1}$
Boolean	$\{0, 1\}$	$\vee$	$\wedge$	0	1
Counting	$\mathbb{N}$	$+$	$\cdot$	0	1
Tropical	$\mathbb{Z} \cup \{+\infty\}$	min	$+$	$+\infty$	0
Goldilocks	$\mathbb{F}_p$	$+_p$	$\cdot_p$	0	1

B.2 Axiom Verification Methodology

We verify semiring axioms at two levels:

Lean 4 formalization. The `KleeneSemiring` typeclass axiomatizes commutativity, associativity, distributivity, identity, and annihilation. Semiring axioms for Boolean, counting, and Goldilocks types are machine-checked. The Kleene star operation is formalized but applied only where the infinite sum converges (e.g., $0^* = 1$ for counting).

Property-based testing. Each axiom is tested via Rust `proptest` with 256 random inputs per semiring type. An independent Python test suite re-verifies on 500 random trials, providing cross-language validation.

Definition B.1 (Embeddable semiring)**.** *A semiring* $(\mathcal{S}, \oplus, \otimes, \bar{0}, \bar{1})$ *is* $\mathbb{F}_p$-embeddable *if there exists an injective map* $\phi : \mathcal{S} \to \mathbb{F}_p$ *satisfying* $\phi(\bar{0}) = 0$*,* $\phi(\bar{1}) = 1$*,* $\phi(a \oplus b) = \phi(a) + \phi(b)$*, and* $\phi(a \otimes b) = \phi(a) \cdot \phi(b)$*.*

Proposition B.2 (Boolean embedding validity for deterministic WFA)**.** *In a deterministic WFA, for each state* q *and input symbol* σ*, at most one outgoing transition has weight* 1*. Consequently, the inner sum* $\bigoplus_i v_i \otimes \mu_\sigma[i,j] = \bigvee_i (v_i \wedge \mu_\sigma[i,j])$ *has at most one non-zero term, and the* $\vee$*-idempotency mismatch (*$\phi(1 \vee 1) = 1 \neq 2$*) never arises.*

C Full Benchmark Data

C.1 Detailed Benchmark Scores

Table 8 provides the complete benchmark evaluation data from §7.

Table 8: Full benchmark evaluation results.

Category	**Pairs**	**EM**	**Tok. F1**	**BLEU**	**R-1**	**R-L**	**Disagree.**
MMLU-style	30	0.600	0.600	0.940	0.600	0.600	0
SQuAD-style	20	0.050	0.600	0.320	0.600	0.580	0
Translation	15	0.000	0.620	0.360	0.620	0.600	0
Random stress	500	0.002	0.120	0.220	0.120	0.110	0
Total	**565**	—	—	—	—	—	**0**

With 5 metrics per pair across 565 unique pairs, plus additional pairs from the 10-seed compilation test (1,015 total), the 2,825 triple-agreement checks all show exact agreement between reference, WFA, and circuit implementations.

C.2 Compilation Correctness by Seed

Each of 10 random seeds produces an independent set of compilation correctness checks. The 57,518 total comprises:

- 50,750 triple-agreement checks (10 seeds × 5 metrics × 1,015 pairs)
- 5,075 score-range validity checks
- 1,524 reflexivity checks (EM, F1, BLEU on identical pairs)
- 169 edge-case checks (8 cases × 10 seeds, plus cross-metric)

C.3 STARK Proof Detailed Data

C.4 Tier 1 Soundness Proof

Trace layout. The execution trace $\mathsf{Tr} \in \mathbb{F}_p^{T \times w}$ has columns: $\mathbf{s}_0, \ldots, \mathbf{s}_{n-1}$ (current state weights), $\mathbf{s}'_0, \ldots, \mathbf{s}'_{n-1}$ (next state weights), σ (input symbol), and auxiliary bookkeeping columns.

Table 9: Complete STARK proof measurements.

Circuit	Prove (ms)	Verify (ms)	Size (B)	Size (KiB)	Valid
fibonacci_8	101	5	36,440	35.6	✓
fibonacci_16	238	9	57,176	55.8	✓
counter_8	193	5	36,160	35.3	✓
counter_16	155	8	56,848	55.5	✓
squaring_8	15	5	36,160	35.3	✓
exact_match_equal (5)	741	5	36,928	36.1	✓
exact_match_differ (5)	261	5	36,928	36.1	✓
exact_match_sentence (22)	101	11	72,992	71.3	✓
token_count_8	121	5	36,416	35.6	✓
Mean	214	6.5	45,005	44.0	9/9
Min	15	5	36,160	35.3	—
Max	741	11	72,992	71.3	—

Constraints.

$$\text{Transition:} \quad s'_j = \sum_{i=0}^{n-1} s_i \cdot \phi(\mu_\sigma[i,j]) \tag{11}$$

$$\text{Initial boundary:} \quad s_i\big|_{t=0} = \phi(\alpha_i) \tag{12}$$

$$\text{Final boundary:} \quad \text{output} = \sum_{i=0}^{n-1} s_i\big|_{t=T} \cdot \phi(\eta_i) \tag{13}$$

Proof of Theorem 5.1. By induction on t. After processing $\sigma_1 \cdots \sigma_t$, the state vector satisfies $s_i|_t = \phi(v_i^{(t)})$ where $v^{(t)} = \alpha \otimes \mu_{\sigma_1} \otimes \cdots \otimes \mu_{\sigma_t}$.

Base case ($t = 0$): Boundary constraint (12) sets $s_i|_0 = \phi(\alpha_i) = \phi(v_i^{(0)})$.

Inductive step: By the transition constraint (11) and the homomorphism property of ϕ:

$$s_j|_{t+1} = \sum_i \phi(v_i^{(t)}) \cdot \phi(\mu_{\sigma_{t+1}}[i,j]) = \phi\left(\bigoplus_i v_i^{(t)} \otimes \mu_{\sigma_{t+1}}[i,j]\right) = \phi(v_j^{(t+1)})$$

Final readout: At $t = T$: output $= \sum_i \phi(v_i^{(T)}) \cdot \phi(\eta_i) = \phi([\![\mathcal{A}]\!](w))$. □

C.5 Tier 2 Soundness Details

Comparison gadget. To compute $\min(a, b)$ over $\mathbb{F}_p$ for $a, b \in [0, 2^{63})$:

1. **Bit decomposition**: 64 auxiliary columns per operand with Boolean constraints $a_i(a_i - 1) = 0$.
2. **Carry-chain comparison**: Borrow chain from MSB determines $\mathsf{lt}(a, b) = 1$ iff $a < b$.
3. **Min selector**: $\min(a, b) = \mathsf{lt}(a, b) \cdot a + (1 - \mathsf{lt}(a, b)) \cdot b$.

Lemma C.1 (Comparison gadget soundness)**.** *For $a, b \in [0, 2^{63})$, the comparison gadget correctly computes $\mathsf{lt}(a, b)$ and $\min(a, b)$ over $\mathbb{F}_p$.*

Lemma C.2 (Comparison gadget completeness)**.** *For every $a, b \in [0, 2^{63})$, there exists a unique valid assignment to the auxiliary columns of the comparison gadget.*

C.6 Boolean Embedding

The Boolean semiring $(\{0,1\}, \vee, \wedge, 0, 1)$ embeds into $\mathbb{F}_p$ via $\phi(0) = 0$, $\phi(1) = 1$. The multiplicative homomorphism holds, but $\vee$ does not map to field addition in general ($\phi(1 \vee 1) = 1 \neq 2$). Proposition B.2 shows this mismatch never arises in deterministic WFAs, where at most one incoming transition carries weight 1 for any state–symbol pair.